



Análise e Desenvolvimento de Sistemas **PROGRAMAÇÃO**
ORIENTADA A OBJETOS COM C# RELATÓRIO DE AULAS
PRÁTICAS

Nome: Vitória Freitas

RA: 2523279

Polo de matrícula: Carapicuíba - Vila Dirce

Local da realização da Aula Prática: Alphaville - Santana de Parnaíba

Ano da postagem: 2026

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): CONSTRUINDO UM PROGRAMA DE CONSOLE EM C# 14, COM .NET 10: TIPOS, NULLABILIDADE, PATTERN MATCHING, RECORDS E PROPRIEDADES

Programação Orientada a Objetos (POO)

A Programação Orientada a Objetos é um paradigma de programação que trabalha com sistemas de entidades e objetos. Os 4 pilares da POO são:

- Abstração: retirar entidades do mundo real e abstrair estes itens nos códigos.
- Encapsulamento: garantir que a alteração de alguns atributos dos objetos são exclusivas de métodos e permitir apenas a visualização.
- Herança: uma entidade pode herdar atributos e métodos de outra entidade.
- Polimorfismo: permite que um mesmo método tenha formas diferentes entre as classes.

As classe, objeto, propriedade, método e encapsulamento podem parecer de diversas formas em programas simples, um exemplo, seria um programa que guarda registros de carros de diversas marcas, teria uma classe chamada 'Carro', contendo propriedades (como: 'placa', 'marca', 'kilometros_rodados' e 'preco') e metodos (como: 'alterar_preco', 'acessar_preco' e 'vender'). Cada carro dentro do sistema seria uma instância da classe 'Carro', assim criando diversos objetos, o encapsulamento seria alcançado ao privar de alterar o preço e apenas permitir a visualização dele, para alterar seria através do método, com isso teríamos um programa simples que possui classe, objeto, propriedade, método e encapsulamento.

Tipos por valor e tipos por referência

Tipos por valor são aqueles que armazenam diretamente os seus dados, eles contém em si o valor atribuído, os tipos: int, double, bool, char e struct são tipos por valor. Os tipos por referência armazenam endereços para os objetos alocados no heap, podemos citar como exemplo deles: Class, array e string.

Nullabilidade

A nulabilidade está presente em C# presentes nos tipos anuláveis por valor, eles usam uma sintaxe de interrogação, por exemplo, int?. Assim um tipo por valor pode conter um valor null.

Pattern matching

Pattern matching é uma técnica de testar uma expressão para verificar se ele possui determinadas características, provendo meios com uma sintaxe mais consistente para realização de testes de expressão e quando a expressão passar

nesse teste o programa realizará uma ação. Exemplo:

```
int? valor = 10;
if (valor is not null)
{
    Console.WriteLine($"O valor é igual a {valor}");
};
```

Records e with-expression

O record prover funcionalidade de encapsulamento dos dados de maneira embutida. Quando se declara um construtor primário no record o compilador irá gerar propriedades públicas para os parâmetros dele. Quando define uma classe como record isso significa que os objetos desse tipo serão tratados com valores imutáveis, exemplo:

```
public record Carro
{
    public string Marca { get; init; }
    public string Placa { get; init; }
};
```

O record fornece suporte para a with-expression, com ela podemos criar cópias do objeto com propriedades diferentes, exemplo:

```
var carro1 = new Carro
{
    Marca = "FIAT",
    Placa = "5435-3255"
};
var carro2 = carro1 with { Placa = "4324 - 4317" };
```

Propriedade com validação

Com as propriedades com validação podemos verificar um valor antes de atribuir um valor a ele, caso recebamos um valor null podemos retornar um ArgumentNullException. E também podemos encapsular os valores em campos privados para garantir uma integridade ao dados como o exemplo a seguir:

```
private int _saldo;
public int Saldo
{
    get => _saldo;
```

```

        set {
            if (value < 0) throw new ArgumentException("Valor invalido");
            _saldo = value;
        }
    }
}

```

Atribuição condicional nula

A atribuição condicional nula permite que uma atribuição ocorra condicionalmente, como nesse exemplo:

```

void Exemplo(Valor? v)
{
    v?.Numero = 20; // caso não seja nulo, o valor será atribuído
}

```

Código-fonte comentado

```

public record Produto
{
    public int Id { get; init; }
    public string Nome { get; init; }
    public double Preco { get; init; }
};

public class Config
{
    private string _tema;
    public string Tema
    {
        get => _tema;

        set
        {
            if (value == null) throw new ArgumentNullException("Valor nulo é inválido.");
            _tema = value;
        }
    }
}

```

```

}
public class Cliente
{
    public decimal Total { get; set; }
}
namespace relatorio1
{
    class Program
    {
        static void Main()
        {
            //Etapa - 1: Imprimindo uma saldação na tela
            Console.WriteLine("Olá, bom dia");
            //Etapa - 2: Declarando variáveis tipos por valor
            int idade = 19;
            string nome = "Isaac";
            Console.WriteLine($"Olá, meu nome é {nome} e tenho {idade} anos.");
            //Etapa 3 - Nullabilidade
            string? frase = null;
            string exibir = frase ?? "Valor não atribuído";
            Console.WriteLine(exibir);
            frase ??= "O dia está ótimo";
            Console.WriteLine(frase);
            //Etapa 4 - switch como expressão
            object? valor = 120;
            string descricao = valor switch
            {
                null => "Valor nulo",
                string => "Valor é uma string",
                int x when x > 100 => "Valor inteiro maior que 100",
                _ => "Valor não encontrado"
            };
            Console.WriteLine(descricao);
        }
    }
}

```

//Etapa 5 - Função local static

```
static decimal Calc(decimal preco, decimal valor = 0.12m) => preco * valor;
```

```
decimal resultado = Calc(100.2m);
```

```
Console.WriteLine(resultado);
```

//Etapa 6 - record

```
var p1 = new Produto
```

```
{
```

```
    Id = 1,
```

```
    Nome = "Celular",
```

```
    Preco = 1240.05
```

```
};
```

```
var p2 = p1 with { Preco = 1300.00 };
```

```
Console.WriteLine(p1);
```

```
Console.WriteLine(p2);
```

//Etapa 7 - Classe com propriedade e o token contextual field

```
var newConfig = new Config();
```

```
newConfig.Tema = "Azul";
```

```
Console.WriteLine(newConfig.Tema);
```

//Etapa 8 - Classe Cliente e atribuição condicional com ?. no destino

```
Cliente? cliente = new Cliente();
```

```
cliente?.Total += 100.50m;
```

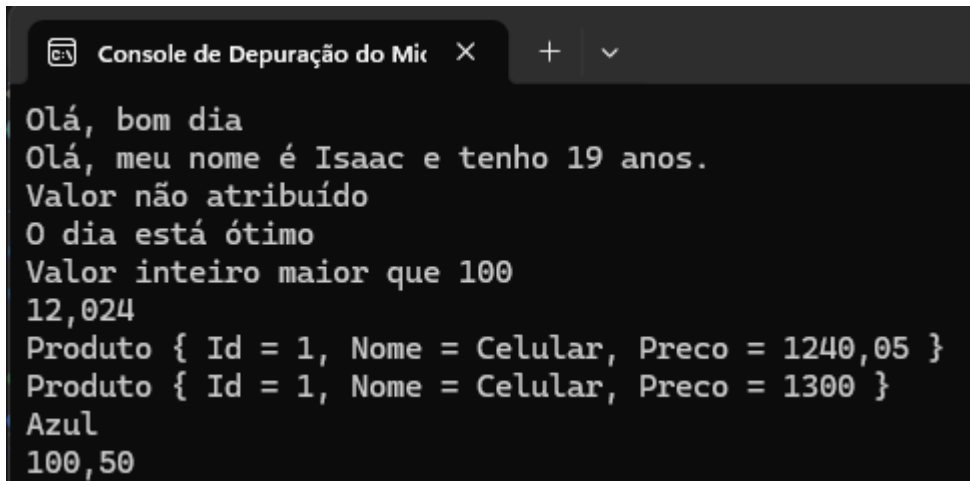
```
Console.WriteLine(cliente?.Total);
```

```
}
```

```
}
```

```
}
```

Resultado no terminal:



```
Olá, bom dia
Olá, meu nome é Isaac e tenho 19 anos.
Valor não atribuído
O dia está ótimo
Valor inteiro maior que 100
12,024
Produto { Id = 1, Nome = Celular, Preço = 1240,05 }
Produto { Id = 1, Nome = Celular, Preço = 1300 }
Azul
100,50
```

Referências

DIO. **POO: programação orientada a objetos – guia completo**. Disponível em: <https://www.dio.me/articles/poo-programacao-orientada-a-objetos-guia-completo>. Acesso em: 24 abr. 2026.

UNIVERSIDADE PAULISTA (UNIP). **Livro-texto – Unidade I: Programação Orientada a Objetos com C#**. Disponível em: https://ava.ead.unip.br/bbcswebdav/pid-5065360-dt-content-rid-17960334_1/institution/Conte%C3%BAdos_AVA/DISCIPLINAS_GERAIS/8120-60%20-%20Programa%C3%A7%C3%A3o%20Orientada%20a%20Objetos%20com%20C%23/LibroTexto%20-%20Unidade%20I.pdf. Acesso em: 24 abr. 2026.

MICROSOFT. **Pattern matching in C#**. Disponível em: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/functional/pattern-matching>. Acesso em: 24 abr. 2026.

MICROSOFT. **Record types in C#**. Disponível em: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/record>. Acesso em: 24 abr. 2026.

MACORATTI, José Carlos. **C# 9.0 – usando records (parte 1)**. Disponível em: https://www.macoratti.net/20/11/c9_record1.htm. Acesso em: 24 abr. 2026.

MICROSOFT. **Usando propriedades (C# Programming Guide)**. Disponível em: <https://learn.microsoft.com/pt-br/dotnet/csharp/programming-guide/classes-and-structs/using-properties>. Acesso em: 24 abr. 2026.

MICROSOFT. **Null-conditional assignment (C# 14.0 proposal)**. Disponível em: <https://learn.microsoft.com/pt-br/dotnet/csharp/language-reference/proposals/csharp-14.0/null-conditional-assignment>. Acesso em: 24 abr. 2026.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): DESENVOLVENDO UM PROGRAMA DE CONSOLE EM C#: AS CONDICIONAIS E OS LAÇOS (COM .NET 10 E C# 14)

Instruções condicionais

Estruturas de controles, como if e else permitem que o programa faça ações baseadas em condições, caso seja verdadeira ele vai tomar uma ação, caso seja falsa tomará outra diferente, assim controlando o fluxo do programa.

O que é o switch

O switch é uma estrutura condicional para selecionar um caso entre várias opções de blocos de códigos que podem ser executadas, dependendo do resultado da expressão ele irá executar um deles. Estrutura em c#:

```
switch (expression)
{
    case x:
        // código do bloco
        break;
    case y:
        // código do bloco
        break;
    default:
        // código do bloco
        break;
}
```

Pode ser utilizado no lugar de if e else encadeados, trazendo uma melhor legibilidade e organização para código.

O que são os laços de repetição (for e while)

Laços de repetição são estruturas de condição que vão executar repetidamente um bloco de código até uma determinada expressão for verdadeira. O while é ideal quando não sabemos quantas vezes o laço irá repetir, o for é ideal

quando o laço será executado uma determinada quantidade de vezes.

Case-sensitive

Case-sensitive é quando um programa é sensível a letras minúsculas e maiúsculas, portanto uma variável X é uma variável x serão consideradas diferentes para o programa.

O papel de ponto e vírgula (;) e de chaves ({ e }) na estrutura do código.

O papel do ponto e vírgula ele diz quando uma linha de código acabou e as chaves para agrupar diversos comandos em bloco de código único.

Como métodos e classes podem organizar um programa que inicialmente foi escrito no arquivo principal.

Com os métodos e classes podemos organizar as funções do programa de uma melhor maneira, assim cada método cumprirá uma funcionalidade diferente.

O código-fonte comentado

Program.cs

```
namespace relatorio2;
class Program
{
    static void Main(string[] args)
    {
        SystemClass system = new SystemClass();
        //Etapa 1 — Mensagens iniciais no console
        Console.WriteLine("Demonstração de condicionais e laços");
        Console.WriteLine();
        //Etapa 2 — Identificadores case -sensitive
        system.CaseSensitiveCase();
        //Etapa 3 — Classificação por nota com if / else if / else | Testamos com esses
        //números: 40, 70 e 85
        system.GradeClassifier(85);
        //Etapa 4 — Comando com switch | Testamos com esses comandos: A, B, C e D
        system.SwitchCommand('A');
        //Etapa 5 — Contagem com o laço for | Testamos com esses números: 5, 10, 15
        system.Count(5);
        //Etapa 6 — Contagem regressiva com o laço while | Testamos com esses
        //números: 3, 5, 10
        system.RegressiveCount(3);
        string? entrada = null;
        do
        {
            Console.WriteLine("\nAperte enter para encerrar o console.");
            entrada = Console.ReadKey().Key.ToString();
        } while (entrada != "Enter");
    }
}
```

```
}
```

SystemClass.cs

```
namespace relatorio2
{
    public class SystemClass
    {
        public void CaseSensitiveCase()
        {
            int aluno = 10;
            int Aluno = 20;
            Console.WriteLine($"aluno: {aluno}");
            Console.WriteLine($"Aluno: {Aluno}");
        }
        public void GradeClassifier(int nota)
        {
            if (nota < 50)
            {
                Console.WriteLine("Desempenho ruim");
            }
            else if (nota < 70)
            {
                Console.WriteLine("Desempenho mediano");
            }
            else if (nota < 90)
            {
                Console.WriteLine("Desempenho bom");
            }
            else
            {
                Console.WriteLine("Desempenho sensacional");
            }
        }
        public void SwitchCommand(char comando)
        {
            //Expressão switch, guardando resultado em variável
            /*
            string mensagem = comando switch
            {
                'A' => "Comando A",
                'B' => "Comando B",
                'C' => "Comando C",
                _ => "Comando desconhecido"
            };
            Console.WriteLine(mensagem);
            */
            switch (comando)
            {
            }
```

```

        case 'A':
            Console.WriteLine("Comando A");
            break;
        case 'B':
            Console.WriteLine("Comando B");
            break;
        case 'C':
            Console.WriteLine("Comando C");
            break;
        default:
            Console.WriteLine("Comando desconhecido");
            break;
    }
}

public void Count(int value)
{
    Console.WriteLine($"Contar até {value}");
    for (var i = 1; i <= value; i += 1)
    {
        Console.WriteLine(i);
    }
}

public void RegressiveCount(int contador)
{
    Console.WriteLine("Contagem regressiva:");
    while (contador >= 1)
    {
        Console.WriteLine(contador);
        contador -= 1;
    }
    Console.WriteLine("Contagem encerrada.");
}
}
}

```

Resultado no terminal:

```
Console de Depuração do Mic X + v
Demonstração de condicionais e laços
aluno: 10
Aluno: 20
Desempenho bom
Comando A

Contar até 5
1
2
3
4
5

Contagem regressiva:
3
2
1
Contagem encerrada.

Aperte enter para encerrar o console.
```

Referências (ABNT)

DIO. **Operadores condicionais em C#: como o if, if aninhado e switch moldam o fluxo do seu programa para respostas inteligentes.** Disponível em: <https://www.dio.me/articles/operadores-condicionais-em-c-como-o-if-if-aninhado-e-switch-moldam-o-fluxo-do-seu-programa-para-respostas-inteligentes>. Acesso em: 27 abr. 2026.

W3SCHOOLS. **C# Switch Statement.** Disponível em: https://www-w3schools-com.translate.goog/cs/cs_switch.php?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc. Acesso em: 27 abr. 2026.

DIO. **Laços de repetição.** Disponível em: <https://www.dio.me/articles/lacos-de-repeticao> . Acesso em: 27 abr. 2026.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): IMPLEMENTANDO UM MINIJOGO DE BATALHA EM CONSOLE COM POO EM C# (CLASSES E OBJETOS)

Definição de classe e de objeto (instância)

A classe atua como um modelo para criação de objetos, servindo como uma estrutura para eles, ela define propriedades e métodos que os objetos irão possuir. Um objeto é uma instância da classe, possuindo valores próprios para os atributos da classe, podendo chamar os métodos da classe.

Nesse método da classe Jogador do mini jogo que criei mostra um exemplo de interação entre objetos no minijogo, um o objeto irá chamar o método de outro para alterar o estado dentro dele:

```
public string Atacar(Inimigo alvo)
{
    if (Vida <= 0) return "\nAtaque do jogador impedido, vida está zerada.";
    if (alvo.Vida <= 0) return "\nAtaque impedido, inimigo sem vida.";
    alvo.ReceberDano(Forca);
    return $" \nAtaque do {Nome} realizado.";
}
```

Propriedades, métodos, construtores, encapsulamento e propriedade calculada em C#.

As Propriedades de uma classe, permitem ler, gravar ou modificar o valor de um atributo da classe, se ele for um campo privado podemos fazer validações, assim as propriedades encapsulam o acesso ao campo privado através de get (retorna o valor da propriedade) e private set (atribui o valor da propriedade). Abaixo um exemplo de uma propriedade calculada:

```
//Propriedade calculada, se a vida for maior que zero, EstaVivo é true, se não é false
public bool EstaVivo { get { return Vida > 0 ? true : false; } }
```

O encapsulamento esconde detalhes internos da classe, proibindo a alteração da propriedade diretamente e permitindo apenas a visualização.

Os métodos são blocos de código que realizam uma ação, podendo retornar algum valor e receber parâmetros tal qual uma função.

Os construtores servem para inicializar uma instância da classe, atribuindo valores aos seus atributos, exemplo de um construtor em c#:

```
public Jogador(string nome, int vida_inicial, int forca)
{
    Nome = nome;
    Vida = vida_inicial;
    Forca = forca;
    Pontuacao = 0;
}
```

Descrição do uso de condicionais e de laços, com foco no loop de turnos da batalha.

Com o uso de condicionais e laços dentro do mini jogo podemos criar uma lógica, na qual a batalha apenas irá se encerrar quando o inimigo ou o jogador estiverem com a vida zerada, segue a parte do código-fonte em que a ela foi implementada:

```
// Loop até algum deles morrer.
while (jogador.EstaVivo == true && inimigo.EstaVivo == true)
{
    Console.WriteLine($"--- Turno: {turno} --- \n");
    string ataqueJogador = jogador.Atacar(inimigo);
    string ataqueInimigo = inimigo.Atacar(jogador);
    Console.WriteLine("Ataque Jogador: " + ataqueJogador);
    Console.WriteLine("Ataque Inimigo: " + ataqueInimigo);

    Console.WriteLine("\n--- Aperte ENTER para continuar ---\n");
    string key = Console.ReadKey().Key.ToString().ToLower();
    if (key != "enter") break;
    turno += 1;
}
```

Código-fonte completo

```
// Program.cs
namespace relatorio3
{
    class Program
    {
        static void Main(string[] args)
        {
            // Criação do jogo
            Jogo jogo = new Jogo();
            // Execução do jogo
            jogo.Execucar();
        }
    }
}
```

```
//Jogador.cs
namespace relatorio3
{
    public class Jogador
    {
```

```

public string Nome { get; set; }
public int Vida { get; private set; }

public int Forca { get; set; }

public int Pontuacao { get; private set; }
//Propriedade calculada, se a vida for maior que zero, EstaVivo é true, se não
é false
public bool EstaVivo { get { return Vida > 0 ? true : false; } }
public Jogador(string nome, int vida_inicial, int forca)
{
    Nome = nome;
    Vida = vida_inicial;
    Forca = forca;
    Pontuacao = 0;
}

public string Atacar(Inimigo alvo)
{
    if (Vida <= 0) return "\nAtaque do jogador impedido, vida está zerada.";
    if (alvo.Vida <= 0) return "\nAtaque impedido, inimigo sem vida.";
    alvo.ReceberDano(Forca);
    return $" \nAtaque do {Nome} realizado.";
}

public string ReceberDano(int dano)
{
    Vida -= dano;
    if (Vida < 0) Vida = 0;
    Console.WriteLine($" \nVida do Jogador {Nome}: {Vida} \n");
    GanharPontos(10);
    return "Dano sofrido.";
}

public void GanharPontos(int pontos)
{
    Pontuacao += pontos;
    Console.WriteLine($" \nPontos do {Nome}: {Pontuacao}");
}
}
}

```

```

//Inimigo.cs
namespace relatorio3
{
    public class Inimigo
    {
        public string Nome { get; set; }
        public int Vida { get; private set; }
    }
}

```

```

public int Forca { get; set; }

public int PontosDeRecompensa { get; private set; }

public bool EstaVivo { get { return Vida > 0 ? true : false; } }

public Inimigo(string nome, int vida_inicial, int forca, int
pontosDeRecompensa)
{
    Nome = nome;
    Vida = vida_inicial;
    Forca = forca;
    PontosDeRecompensa = pontosDeRecompensa;
}

public string Atacar(Jogador alvo)
{
    //Ataque impedido, caso o jogador estiver sem vida ou o inimigo
    if (Vida <= 0) return "\nAtaque do inimigo impedido, vida está zerada.";
    if (alvo.Vida <= 0) return "\nAtaque impedido, alvo sem vida.";
    //Dano no alvo
    alvo.ReceberDano(Forca);
    return $" \nAtaque do {Nome} realizado.";
}

public string ReceberDano(int dano)
{
    Vida -= dano;
    //Se a vida ficar negativa, será mudada para zero
    if (Vida < 0) Vida = 0;
    Console.WriteLine($" \nVida atual {Nome}: {Vida} \n");
    return "Dano sofrido.";
}
}
}

```

```

//Jogo.cs
namespace relatorio3
{
    public class Jogo
    {
        public void Execucar()
        {
            Jogador jogador1 = new Jogador("Link", 130, 30);
            Inimigo inimigo1 = new Inimigo("Slime", 120, 10, 15);
            Inimigo inimigo2 = new Inimigo("Goblin", 110, 12, 12);
            Inimigo inimigo3 = new Inimigo("Moblin", 130, 14, 12);
        }
    }
}

```



```

// Lista de inimigos que o jogador enfrentará
List<Inimigo> inimigos = new List<Inimigo> { inimigo1, inimigo2, inimigo3
};

Console.WriteLine($"Jogador Estados Iniciais: ");
Console.WriteLine($"Nome: {jogador1.Nome} - Vida: {jogador1.Vida}");

Console.WriteLine($"Inimigo 1 Estados Iniciais: ");
Console.WriteLine($"Nome: {inimigo1.Nome} - Vida: {inimigo1.Vida}");

Console.WriteLine($"Inimigo 2 Estados Iniciais: ");
Console.WriteLine($"Nome: {inimigo2.Nome} - Vida: {inimigo2.Vida}");

// Para cada inimigo ocorrerá uma batalha, se o jogador estiver vivo
inimigos.ForEach(x =>
{
    if (jogador1.EstaVivo == true) Batalhar(jogador1, x);
}
);

Console.WriteLine("Batalha encerrada.");

Console.WriteLine($"Status do Jogador Finais: ");
Console.WriteLine($"Vida: {jogador1.Vida} - Pontos
{jogador1.Pontuacao}");
}

public void Batalhar(Jogador jogador, Inimigo inimigo)
{
    Console.WriteLine("=== Batalha ===");
    Console.WriteLine($"Entre {jogador.Nome} - {inimigo.Nome}");
    Console.WriteLine($"Status antes da batalha");
    Console.WriteLine($"Jogador: {jogador.Nome} - Vida: {jogador.Vida} -
Ataque: {jogador.Forca}");
    Console.WriteLine($"Inimigo: {inimigo.Nome} - Vida: {inimigo.Vida} -
Ataque: {inimigo.Forca}");

    int turno = 1;
    // Loop até algum deles morrer.
    while (jogador.EstaVivo == true && inimigo.EstaVivo == true)
    {
        Console.WriteLine($"--- Turno: {turno} ---");
        string ataqueJogador = jogador.Atacar(inimigo);
        string ataqueInimigo = inimigo.Atacar(jogador);
        Console.WriteLine("Ataque Jogador: " + ataqueJogador);
        Console.WriteLine("Ataque Inimigo: " + ataqueInimigo);

        Console.WriteLine("--- Aperte ENTER para continuar ---");
        string key = Console.ReadKey().Key.ToString().ToLower();
        if (key != "enter") break;
    }
}

```

```

        turno += 1;
    }

    //Se o jogador estiver vivo e o inimigo não ele foi o vencedor.
    if(jogador.EstaVivo == true && inimigo.EstaVivo == false)
    {
        Console.WriteLine($"{jogador.Nome} vencedor!");
        jogador.GanharPontos(10);
    }
    else
    {
        Console.WriteLine("Fim de jogo, vida do jogador zerada.");
    }
}
}
}

```

Teste com o dano do inimigo sendo maior que a vida do jogador (teste para verificar que a vida não fica negativa):

```

Jogador Estados Iniciais:
Nome: Link - Vida: 130
Inimigo 1 Estados Iniciais:
Nome: Slime - Vida: 120
Inimigo 2 Estados Iniciais:
Nome: Goblin - Vida: 110
=== Batalha ===
Entre Link - Slime
Status antes da batalha
Jogador: Link - Vida: 130 - Ataque: 30
Inimigo: Slime - Vida: 120 - Ataque: 140

--- Turno: 1 ---

Vida atual Slime: 90

Vida do Jogador Link: 0

Pontos do Link: 10
Ataque Jogador:
Ataque do Link realizado.
Ataque Inimigo:
Ataque do Slime realizado.

--- Aperte ENTER para continuar ---

Fim de jogo, vida do jogador zerada.
Batalha encerrada.
Status do Jogador Finais:
Vida: 0 - Pontos 10

```

Teste com uma batalha completa:

```
=== Batalha ===  
  
Entre Link - Slime  
  
Status antes da batalha  
Jogador: Link - Vida: 130 - Ataque: 30  
Inimigo: Slime - Vida: 120 - Ataque: 10
```

```
Link vencedor!  
  
Pontos do Link: 40
```

Teste com uma segunda batalha:

```
=== Batalha ===  
  
Entre Link - Goblin  
  
Status antes da batalha  
Jogador: Link - Vida: 100 - Ataque: 30  
Inimigo: Goblin - Vida: 110 - Ataque: 12
```

```
Link vencedor!  
  
Pontos do Link: 80
```

Referências

CENDRON, Marcelo. **Linguagem Python – 08/01**. Instituto Federal Catarinense, [s.d.]. Disponível em: https://userdir.luzerna.ifc.edu.br/~marcelocendron/tutoriais/linguagem_python/08/01.php. Acesso em: 29 abr. 2026.

NEXT WAVE EDUCATION. **Encapsulamento em C# utilizando get e set**. Next Wave Education, [s.d.]. Disponível em: <https://nextwave.education/encapsulamento-em-c-utilizando-get-e-set/>. Acesso em: 29 abr. 2026.

VIEIRA, Eucimar. **Propriedades, métodos e construtores com C#**. DIO, 20 jul. 2025. Disponível em: <https://www.dio.me/articles/propriedades-metodos-e-construtores-com-c>. Acesso em: 29 abr. 2026.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): IMPLEMENTANDO UMA CONTA BANCÁRIA COM POO EM C# (ENCAPSULAMENTO, PROPRIEDADES E MÉTODOS)

O que é POO?

O significado de POO é programação orientada a objetos, ela consiste em ter objetos que contenham dados e métodos, quem vai dar essa estrutura para eles são as classes. Elas contém o modelo para os objetos seguirem ao serem instanciados de uma delas, reunindo características e comportamentos em que eles irão possuir. O objeto ele qualifica os atributos da classe e executa os métodos dela.

Encapsulamento

O encapsulamento é a prática dentro da programação orientada a objetos que oculta informações internas da classe, permitindo apenas visualização e negando a modificação diretamente, apenas deixando modificar através de métodos, assim expondo apenas o necessário. Com o encapsulamento detalhes de implementação de um objeto podem ser modificados sem afetar o restante do programa.

A diferença entre campo e propriedade.

Um campo é um membro classe e carregam valores do objeto, uma propriedade carrega uma maneira para ler, escrever ou calcular um valor do objeto.

A explicação de um get público e de um set restrito (por exemplo, private set).

Um get público nos permite ter acesso a visualização por fora da classe e com o private set nos negamos a modificação do valor diretamente.

A justificativa do uso de decimal para valores monetários.

Os tipos float e double representam números baseados na base 2, por essa razão somente esses números serão representados com precisão. O decimal funciona na base 10, assim tendo uma maior precisão com números na base 10, por essa razão o tipo decimal é melhor para cálculos monetários.

Papel de um construtor com valor padrão (parâmetro opcional).

Com um construtor com um valor padrão é essencial para casos que queremos que um atributo da classe tenha um valor predefinido.

Como métodos podem impor regras de negócio.

Com a utilização de métodos podemos criar uma lógica de programação para que uma regra de negócio seja cumprida, como por exemplo garantir que os usuários não possam sacar um valor maior que o seu próprio saldo.

Código-fonte

```
//ContaBancaria.cs
namespace relatorio4
{
    public class ContaBancaria
    {
        //Campo privado
        private decimal _saldo;
        //Propriedade pública que apenas permite visualização, retorna _saldo
        public decimal Saldo { get => _saldo; }

        public ContaBancaria(decimal saldoInicial = 0m)
        {
            _saldo = saldoInicial;
        }

        public bool Depositar(decimal quantia)
        {
            if (quantia <= 0) return false;
            _saldo = quantia;
            return true;
        }

        public bool Sacar(decimal quantia)
        {
            if (quantia <= 0) return false;
            if (quantia > _saldo) return false;
            _saldo -= quantia;
            return true;
        }
    }
}
```

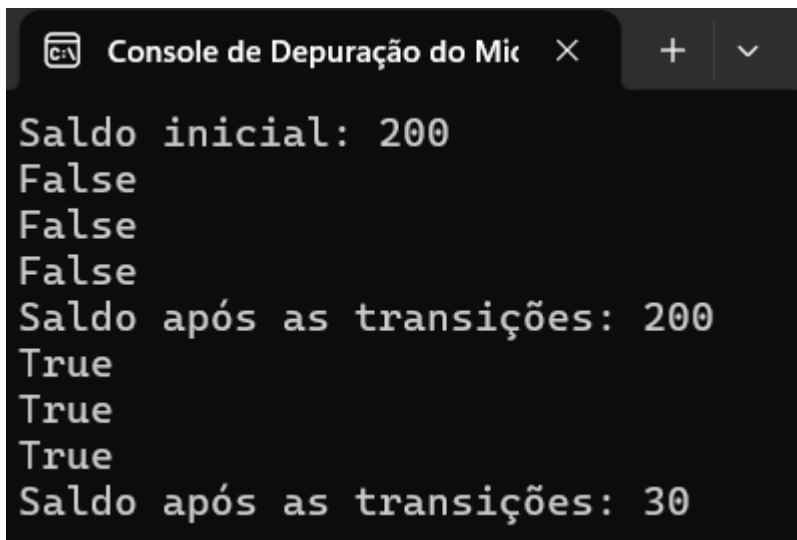
```
//Program.cs
namespace relatorio4
{
    class Program
    {
        static void Main(string[] args)
        {
            ContaBancaria conta = new ContaBancaria(200m);
            Console.WriteLine("Saldo inicial: " + conta.Saldo);
            //Testes com os parâmetros sendo valores inválidos
            bool resultadoDepositoErrado = conta.Depositar(0m);
            bool resultadoSaqueErrado1 = conta.Sacar(0m);
            bool resultadoSaqueErrado2 = conta.Sacar(300m);
            Console.WriteLine(resultadoDepositoErrado + "\n" +
            resultadoSaqueErrado1 + "\n" + resultadoSaqueErrado2);
        }
    }
}
```

```

    Console.WriteLine("Saldo após as transições: " + conta.Saldo);
    //Testes com os parâmetros sendo valores válidos
    bool resultadoDepositoCerto = conta.Depositar(150m);
    bool resultadoSaqueCerto1 = conta.Sacar(20m);
    bool resultadoSaqueCerto2 = conta.Sacar(100m);
    Console.WriteLine(resultadoDepositoCerto + "\n" + resultadoSaqueCerto1
+ "\n" + resultadoSaqueCerto2);
    Console.WriteLine("Saldo após as transições: " + conta.Saldo);
}
}
}

```

Resultado dos testes no console



```

C# Console de Depuração do Mic X + v
Saldo inicial: 200
False
False
False
Saldo após as transições: 200
True
True
True
Saldo após as transições: 30

```

Referências

ALURA. Qual a diferença entre um campo e uma propriedade? [Fórum]. Postado por

Fabiano Augusto Teodoro. 21 nov. 2019. Disponível em:

<https://cursos.alura.com.br/forum/topico-qual-a-diferenca-entre-um-campo-e-uma-propriedade-96546>. Acesso em: 1 maio 2026.

DIO. Classes e objetos na programação orientada a objetos. 29 nov. 2024. Disponível em:

<https://www.dio.me/articles/classes-e-objetos-na-programacao-orientada-a-objetos>.

Acesso em: 1 maio 2026.

LOCAWEB. O que é encapsulamento na programação orientada a objetos. 28 mar. 2023.

Disponível

em:

<https://www.locaweb.com.br/blog/temas/codigo-aberto/o-que-e-encapsulamento/>.

Acesso em: 1 maio 2026.

NIZZOLA, Marcio. Usar decimal ou double no C#? 11 dez. 2021. Disponível em: <https://marcionizzola.medium.com/usar-decimal-ou-double-no-c-eed8feecff61>. Acesso em: 1 maio 2026.

W3SCHOOLS. C# programação orientada a objetos. Traduzido do inglês. Disponível em: https://www-w3schools-com.translate.goog/cs/cs_oop.php?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc. Acesso em: 1 maio 2026.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ABSTRAÇÃO, HERANÇA E POLIMORFISMO EM C# (APLICAÇÃO DE CONSOLE)

O que é Programação Orientada a Objetos (POO) e o que se pretende ao usar POO

A programação orientada a objetos, é sobre ter objetos que contenham dados e métodos, que vão ser estruturados pelas classes. Com elas podemos construir códigos com uma maior organização e com menos repetição.

O que é uma classe abstrata

Uma classe abstrata é uma classe que não pode ser instanciada, ou seja, você não pode criar objetos a partir dela. Ela pode ser herdada por outras classes, servindo como um modelo para essas classes, podendo ter métodos abstratos e comuns, propriedades, indexadores e eventos.

A diferença entre método abstrato e método virtual.

Os métodos virtuais possuem implementação que podem ser sobrepostas, os métodos abstratos não possuem implementação. Exemplo:

```
public abstract class Base {  
    public abstract void MetodoAbstrato(string nome); //não há implementação  
    public virtual void MetodoVirtual(int x) {  
        //faz algo aqui }  
    }
```

O que é herança e quando ela faz sentido.

Herança é quando uma classe herda métodos e atributos da classe pai, faz sentido quando queremos ter uma classe que herda de outra alguns métodos, assim não precisando reescrever o mesmo código, mudando apenas o necessário.

O que é polimorfismo no contexto de “uma variável do tipo base apontando para um objeto do tipo derivado”.

Polimorfismo no contexto de uma variável do tipo base apontando para um objeto do tipo derivado, significa que podemos tratar objetos de maneiras diferentes mesmo herdando da mesma classe pai, por exemplo, temos uma classe pai e temos dois objetos que são de classes herdadas dela, eles podem ter execuções diferentes entre eles dos métodos da classe pai.

O papel de construtores em uma classe base e em uma classe derivada;

Com os construtores a classe base pode ter os mesmo atributos da classe derivada, mas também pode ter outros além dos que são da classe derivada, assim tendo utilidades únicas.

O que é override e por que ele é exigido ao implementar um método abstrato.

Com o override, a classe base pode sobrescrever o método abstrato da classe em que ela herdou.

Código-fonte

```
namespace relatorio5
{
    // Classe abstrata, que servirá de base para as outras
    public abstract class Animal {
        public string Name { get; set; }
        public Animal(string name)
        {
            Name = name;
        }
        public abstract void EmitirSom();
    }
    //Classes que herdam da Animal
    public class Cachorro : Animal
    {
        public string Name { get; set; }
        //Construtor, base faz referencia ao construtor da classe base
        public Cachorro(string name) : base(name)
        {
            Name = name;
        }
        public override void EmitirSom()
        {
            Console.WriteLine($"{Name}: Au au");
        }
    }
    public class Gato : Animal
    {
        public string Name { get; set; }

        public Gato(string name) : base(name)
        {
            Name = name;
        }

        public override void EmitirSom()
        {
            Console.WriteLine($"{Name}: Miau");
        }
    }
    public class Program
    {
        static void Main()
        {

```

```
//Instancias da classes
Cachorro bred = new Cachorro("Bred");
Gato nala = new Gato("Nala");
//Lista com os objetos criados acima
List<Animal> animais = new List<Animal>{ bred, nala};
//impressão do som de cada animal criado
animais.ForEach(animais => animais.EmitirSom());
    }
}
```

Referências

MACORATTI, José Carlos. C# - Classes abstratas, interface e polimorfismo. jun. 2012.
Disponível em: https://www.macoratti.net/12/06/c_caip1.htm. Acesso em: 1 maio 2026.

STACK OVERFLOW. Qual a diferença entre os métodos virtual e abstract? 13 out. 2016.
Disponível em:
<https://pt.stackoverflow.com/questions/158579/qual-a-diferen%C3%A7a-entre-os-m%C3%A9todos-virtual-e-abstract>. Acesso em: 1 maio 2026.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): INTERFACES E POLIMORFISMO EM C# (GEOMETRIA EM APLICAÇÃO DE CONSOLE)

Interface em POO

Uma interface pode ser definida como um contrato com a classe que implementa ela, e essa classe deverá cumprir esse contrato seguindo os métodos que foram definidos pela interface. As interfaces são construídas com de um ou mais métodos que não possuem um corpo, as operações desses métodos deverão ser realizadas pelo a classe que herdar a interface.

Diferença entre interface e classe abstrata

Uma interface não permite implementação de seus métodos, apenas declarações. As classes abstratas permitem que você faça a implementação dos métodos, assim poderão ser executados pelas classes que herdaram da classe abstrata, com isso podemos reduzir a multiplicação de código. Por tanto, caso seja necessário a multiplicação de código com a utilização da interface, uma classe abstrata seria uma melhor escolha para implementar.

O que é implementação de interface em uma classe concreta.

Quando implementamos uma interface em uma classe concreta, devemos seguir os métodos que foram definidos pela interface, seguindo o seu contrato. Quando criamos uma classe concreta e herdamos uma interface nela em C#, usando o Visual Studio a IDE irá gerar um erro, caso não implementarmos todos os métodos da interface.

O que é polimorfismo com interface

A definição de polimorfismo na POO é um mesmo método agir de formas diferentes em diferentes objetos, por tanto, com a utilização de interfaces podemos alcançar o polimorfismo, pois com elas os objetos podem executar métodos de formas variadas.

O papel de propriedades e de construtores para guardar e inicializar os dados (largura, altura, raio);

Com as propriedades e construtores podemos inicializar os valores que vão ser guardados nos campos, assim quando criarmos a lógica dos métodos de cálculo (de largura, altura, raio), podemos utilizar essas propriedades já inicializadas para calcular e retornar o valor do resultado.

A diferença entre área e perímetro, com as fórmulas que você usou no código.

A fórmula utilizada para área de um retângulo foi: base x altura e para círculo foi: π

x r ao quadrado. Para o perímetro de um retângulo foi a soma dos lados, do círculo foi: 2π x r.

Código-fonte

```
//IForma.cs
namespace relatorio6
{
    public interface IForma
    {
        public double CalcularArea();
        public double CalcularPerimetro();
    }
}
```

```
//Program.cs
namespace relatorio6
{
    class Retangulo : IForma
    {
        public double Largura { get; set; }
        public double Altura { get; set; }

        public Retangulo(double largura, double altura)
        {
            Largura = largura;
            Altura = altura;
        }
        public double CalcularArea()
        {
            return Largura * Altura; // fórmula => base x altura
        }
        public double CalcularPerimetro()
        {
            return Largura + Altura + Largura + Altura; // fórmula => soma de todos os
lados
        }
    }

    class Circulo : IForma
    {
        public double Raio { get; set; }

        public Circulo(double raio)
        {
            Raio = raio;
        }
        public double CalcularArea()
        {

```

```

        return Math.PI * Math.Pow(Raio, 2); // fórmula =>  $\pi \times r$  ao quadrado.
    }
    public double CalcularPerimetro()
    {
        return 2 * Math.PI * Raio; // fórmula =>  $2\pi \times r$ .
    }
}
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Insira o valor da largura do retângulo: ");
        string? LarguraRet = Console.ReadLine();
        Console.WriteLine("Insira o valor da altura do retângulo: ");
        string? AlturaRet = Console.ReadLine();
        Console.WriteLine("Insira o valor do raio do círculo: ");
        string? CirculoRaio = Console.ReadLine();
        //If para certificar que os valores recebidos não são nulos
        if (LarguraRet != null && AlturaRet != null && CirculoRaio != null)
        {
            Retangulo retangulo = new Retangulo(double.Parse(LarguraRet),
double.Parse(AlturaRet));
            Circulo circulo = new Circulo(double.Parse(CirculoRaio));
            List<IForma> formas = new List<IForma>{ retangulo, circulo};
            formas.ForEach(x =>
            {
                Console.WriteLine(x.CalcularArea().ToString("F2")); // retorno em 2
casas decimais
                Console.WriteLine(x.CalcularPerimetro().ToString("F2")); // retorno em
2 casas decimais
            });
        }
    }
}
}

```

Referências

DEVMEDIA. Interfaces programação orientada a objetos. Disponível em: <https://www.devmedia.com.br/interfaces-programacao-orientada-a-objetos/18695>. Acesso em: 2 maio 2026.

TABNEWS. OOP: quando usar interface e quando usar uma classe abstrata. Disponível em: <https://www.tabnews.com.br/daniloflorenzano/oop-quando-usar-interface-e-quando-usar-uma-classe-abstrata>. Acesso em: 2 maio 2026.

TODA MATÉRIA. Área do círculo. Disponível em:
<https://www.todamateria.com.br/area-do-circulo/>. Acesso em: 2 maio 2026.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): Modelando herança e polimorfismo com as classes Animal e Cachorro em C#

Classe e de objeto

Classes podem ser vistas como o modelo que os objetos vão seguir, definindo quais propriedades eles possuíram e os seus métodos. Um objeto é uma instância de classe, podendo executar os métodos da classe, quando criamos o objeto podemos definir os valores de suas propriedades através do construtor da classe.

Encapsulamento

O encapsulamento é você ocultar informações internas, apenas permitindo visualização de propriedades privadas e negando a modificação direta. Por exemplo, podemos ter uma propriedade chamada 'Idade' que permite somente a visualização e para mudar a idade se usa o método 'MudarIdade', que antes de realizar a alteração verifica se o valor que será atribuído é válido.

Herança, classe base e classe derivada

A herança é o que permite que uma classe compartilhe com outras seus métodos e seus atributos.

A classe base é a que concede às suas características a outra classe, a classe derivada é a que herda essas características.

Polimorfismo

O polimorfismo é quando uma classe herda características (métodos e atributos) de outra, mas a execução dos métodos ocorre de maneira diferentes, conseguimos alcançar isso através de métodos virtual na classe base e com o override na classe derivada, o método virtual define o método e com override podemos sobrescrever ele e executa-lo de maneira diferente na classe derivada.

Código-Fonte

```
namespace relatorio7
{
    //Classe Base: Animal
    class Animal
    {
        public string Name { get; set; }
```

```

public Animal(string name)
{
    Name = name;
}

public void Comer()
{
    Console.WriteLine($"Animal {Name} está comendo.");
}

public virtual void FazerSom()
{
    Console.WriteLine($"Animal {Name} está fazendo um som.");
}
}

//Classes derivas: Cachorro e Gato
class Cachorro : Animal
{
    public Cachorro(string name) : base(name)
    {
        Name = name;
    }

    public void AbanarRabo()
    {
        Console.WriteLine($"Cachorro {Name} está abanando o rabo.");
    }

    public override void FazerSom()
    {
        Console.WriteLine($"Cachorro {Name} fez um som: au au.");
    }
}

```



```

}

class Gato : Animal
{
    public Gato(string name) : base(name)
    {
        Name = name;
    }

    public void Arranhar()
    {
        Console.WriteLine($"Gato {Name} está arranhando o arranhador.");
    }

    public override void FazerSom()
    {
        Console.WriteLine($"Gato {Name} fez um som: miau.");
    }
}

class Program
{
    static void Main(string[] args)
    {
        //Animal instância
        Animal animal = new Animal("Max");
        //Cachorro instância
        Cachorro cachorro = new Cachorro("Fred");
        //Gato instância
        Gato gato = new Gato("Scar");
        //Lista de animais
        List<Animal> animais = new List<Animal>{ animal, cachorro, gato};
        animal.Comer();
        cachorro.AbanarRabo();
    }
}

```

```
gato.Arranhar();  
//ForEach para executar o método FazerSom em cada objeto tipo Animal  
animais.ForEach(a => a.FazerSom());  
    }  
}  
}
```

Referências

DEV MEDIA. Conceitos e exemplos: herança na programação orientada a objetos – parte 1. Disponível em: <https://www.devmedia.com.br/conceitos-e-exemplos-heranca-programacao-orientada-a-objetos-parte-1/18579>. Acesso em: 2 maio 2026.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): Herança, polimorfismo e casting com Animal e Cachorro em C#

Classe e de objeto

Classes podem ser vistas como uma planta que os objetos vão seguir, definindo suas propriedades e os seus métodos. Quando instalamos uma classe criamos um objeto, assim podemos definir os valores de suas propriedades e executar os seus métodos.

Herança

A classe Cachorro : Animal, ela está herdando as propriedades e métodos da classe Animal, assim não precisamos escrever as mesmas propriedades e métodos da classe Animal, com isso reutilizamos o código e diminuimos a repetição dele.

A explicação de polimorfismo

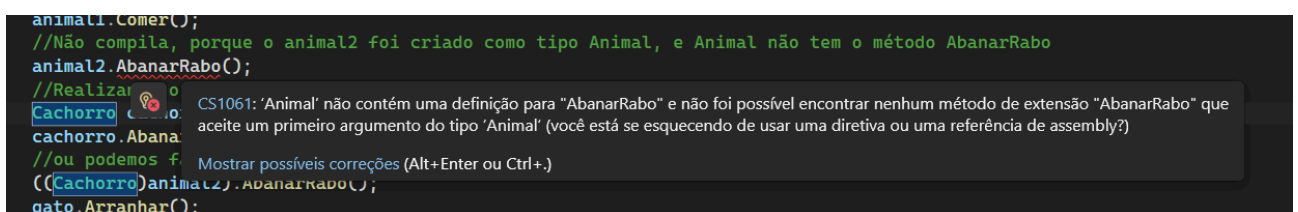
Polimorfismo ocorre quando uma classe herda atributos e métodos de outra, porém pode implementar esses métodos de formas diferentes. Isso é possível utilizando métodos marcados como *virtual* na classe base e a palavra-chave *override* na classe derivada. O método *virtual* define um comportamento padrão, enquanto o *override* permite que esse comportamento seja modificado, fazendo com que o método seja executado de maneira específica na classe derivada.

Como uma variável do tipo Animal pode referenciar um objeto do tipo Cachorro e casting (conversão de tipo)

Quando temos uma classe derivada que herda da classe base, quando instanciamos a derivada, podemos colocar o tipo da variável como o nome da classe base, por exemplo, temos a classe Cachorro que herda da classe Animal, quando instanciamos a classe Cachorro, podemos colocar o tipo como Animal, desta maneira:

```
Animal animal2 = new Cachorro();
```

O problema com isso é caso a classe Cachorro tenha um método que não tem na classe Animal, o compilador gerará um erro, como mostra a imagem abaixo:



```
animal1.Comer();
//Não compila, porque o animal2 foi criado como tipo Animal, e Animal não tem o método AbanarRabo
animal2.AbanarRabo();
//Realizar o
Cachorro o.
cachorro.AbanarRabo();
//ou podemos f
((Cachorro)animal2).AbanarRabo();
gato.Arranhar();
```

Para solucionar esse problema fizemos uma conversão de tipo (casting), que

pode ser feito de duas maneira que gerarão o mesmo resultado no terminal:

```
Cachorro cachorro = (Cachorro)animal2;  
cachorro.AbanarRabo();  
//ou podemos fazer assim:  
((Cachorro)animal2).AbanarRabo();
```

A distinção entre erro de compilação e erro em tempo de execução

Quando ocorre erros de compilação o compilador não permite que o programa seja rodado, por exemplo erros de sintaxe ou semântica. Os erros em tempo de execução ocorrem quando o programa já está executando, como exemplo o programa tentar dividir um número por zero.

Código-fonte

```
namespace relatorio8  
{  
    //Classe Base: Animal  
    class Animal  
    {  
        public string Name { get; set; }  
  
        public void Comer()  
        {  
            Console.WriteLine($"Animal {Name} está comendo.");  
        }  
  
        public virtual void FazerSom()  
        {  
            Console.WriteLine($"Animal {Name} está fazendo um som genérico.");  
        }  
    }  
    //Classes derivas: Cachorro e Gato  
    class Cachorro : Animal  
    {  
        public void AbanarRabo()  
        {  
            Console.WriteLine($"Cachorro {Name} está abanando o rabo.");  
        }  
  
        public override void FazerSom()  
        {  
            Console.WriteLine($"Cachorro {Name} fez um som: au au.");  
        }  
    }  
  
    class Gato : Animal
```

```

{
    public void Arranhar()
    {
        Console.WriteLine($"Gato {Name} está arranhando o arranhador.");
    }

    public override void FazerSom()
    {
        Console.WriteLine($"Gato {Name} fez um som: miau.");
    }
}

class Program
{
    static void Main(string[] args)
    {
        //Animal instância
        Animal animal1 = new Animal();
        animal1.Name = "Max";
        //Cachorro instância
        Animal animal2 = new Cachorro();
        animal2.Name = "Fred";
        //Gato instância
        Gato gato = new Gato();
        gato.Name = "Scar";
        //Lista de animais
        List<Animal> animais = new List<Animal> { animal1, animal2, gato };
        //ForEach para executar o método FazerSom em cada objeto tipo Animal
        animais.ForEach(a => a.FazerSom());
        //Compila, pois Comer existe na classe base
        animal1.Comer();
        //Não compila, porque o animal2 foi criado como tipo Animal, e Animal não tem o
método AbanarRabo
        //animal2.AbanarRabo();
        //Realizando o Casting
        Cachorro cachorro = (Cachorro)animal2;
        cachorro.AbanarRabo();
        //ou podemos fazer assim:
        ((Cachorro)animal2).AbanarRabo();
        gato.Arranhar();
    }
}
}

```

Referências

DEVMEDIA. Conceitos e exemplos: herança na programação orientada a objetos – parte 1. Disponível em:

<https://www.devmedia.com.br/conceitos-e-exemplos-heranca-programacao-orientada-a-objetos-parte-1/18579>. Acesso em: 2 maio 2026.

ROY, Sandip. Runtime vs. compile time. Disponível em: https://www.baeldung-com.translate.goog/cs/runtime-vs-compile-time?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc. Acesso em: 2 maio 2026.